

小学题库项目需求与技术方案研究报告

摘要

本报告面向一个以**自动入库、检索、拍照识题、AI讲题、错题本与个性化练习**为核心的小学题库项目，给出一份适合工程落地的功能与技术需求清单。结论很明确：这个项目的**第一性问题不是“大模型会不会解题”，而是“能不能把混乱资料高质量地结构化进库，并让检索始终先尊重年级、学科、题型、知识点、难度等元数据”**。在搜索层，关键词检索与结构化过滤应当是主干，向量检索与重排是增强；在入库层，OCR、版面解析、切题、结构化抽取、规则校验、置信度和人工审核是决定最终质量的关键。Elastic 官方文档明确指出，`filter` 适合结构化字段且不参与打分；同时 Elasticsearch 支持 `dense_vector`、kNN 过滤与 RRF 混合检索。PaddleX/PaddleOCR 官方文档也表明，其通用版面解析产线已经覆盖版面分析、OCR、表格与公式识别等文档理解能力，非常适合试卷、课本、扫描件这类混合输入。 ¹

假设说明

本报告基于以下未指定信息做工程假设：

- 输入资料格式混合，包含 PDF、Word、Excel、图片、扫描件与手机拍照。
- 样本规模未指定；默认按“中型题库”设计，即原始资料数万到数十万份、结构化题目 10 万到 100 万道可平滑扩展。
- 目标用户为中国大陆小学阶段学习场景，界面语言为简体中文。
- 若项目面向 C 端未成年人，需满足中国关于个人信息、儿童个人信息与未成年人网络保护的要求。

其中，未成年人产品若收集不满十四周岁儿童个人信息，需要监护人同意，并应遵守正当必要、知情同意、目的明确、安全保障等原则；面向未成年人的在线教育网络产品和服务不得推送与教学无关的信息。 ²

推荐的基线架构是：**PostgreSQL 作为业务主库，Elasticsearch 负责关键词与结构化过滤检索，向量层优先选 pgvector 或 Qdrant，OCR 优先用 PaddleOCR/PaddleX，解释与抽取层使用支持图像输入与 Structured Outputs 的 LLM，Codex 只用于研发提效而不作为线上推理引擎**。OpenAI 官方文档显示，Responses API 支持文本与图像输入，Structured Outputs 可强制输出满足 JSON Schema 的结构化 JSON；Codex 则定位为本地/开发环境中的 coding agent，适合代码生成、重构、测试、迁移和调试。 ³

从开发顺序看，最佳路径不是一次性做成“大而全”的学习产品，而是按 **MVP → 自动入库增强 → 检索增强 → 拍照识题 → AI讲题深化 → 错题本与个性化练习** 六个阶段推进。这样做的原因是：题库项目的收益主要来自**数据资产与质量闭环**，而不是聊天能力本身。若自动入库与审核体系没打牢，后续搜题、讲题、错题本和个性化推荐都会建立在低质量数据之上，最终整体体验会系统性失真。 ⁴

功能清单与开发分期

项目功能应围绕五条主链路展开：**资料自动入库链路、检索链路、拍照识题链路、AI讲题链路、学习闭环链路**。其中，自动入库与检索是平台能力；拍照识题与 AI 讲题是交互能力；错题本与个性化练习是留存能力。下面的表格给出“功能 vs 优先级”，其中优先级为本报告的工程建议，而非外部事实判断。

功能域	优先级	目标输出	关键技术要求
自动入库	P0	混合资料自动转为结构化题目	文件解析、OCR、版面分析、切题、结构化抽取、规则校验、置信度、人工审核
检索	P0	按年级/学科/题型/知识点/难度找题	ES 文本检索、 <code>keyword/date</code> 过滤、题目去重、重排、向量召回增强
拍照识题	P1	从手机照片定位到原题/相似题	图像预处理、OCR、版面定位、整页切题、图片与文本双路召回
AI讲题	P1	输出适合小学年级的分步讲解	结构化题目输入、约束式提示、Schema 输出、步骤模板、引用标准答案/解析
错题本	P2	自动收录错题与复习闭环	错因归类、知识点回写、复练调度、掌握度更新
个性化练习	P3	基于薄弱点推荐练习	学生画像、知识点掌握模型、题目约束式推荐、难度与重复控制
智能组卷	P4	条件组卷与阶段测验	题型比例、知识点覆盖、难度分布、去重与曝光控制
家长/教师报告	P5	可解释的学习报告	指标体系、趋势分析、错因分布、可视化与导出

建议的阶段规划如下。这里的人周估算以一个常见的小团队为假设：**后端 1 人、全栈/前端 1 人、算法/OCR/LLM 工程 1 人、测试/产品 0.5-1 人**。估算为经验性工程判断。

阶段	核心目标	关键交付	估算人周
MVP	跑通“资料上传 → 自动入库 → 人工审核 → 正式题库 → 条件检索”	上传、OCR、切题、抽取、审核台、题库管理、ES 基础检索	10-14
阶段 A	做强自动入库质量	多格式解析、表格/公式支持、重复题检测、置信度策略、批量回灌	6-8
阶段 B	做强检索质量	混合检索、向量召回、重排、相似题、搜索分析面板	5-7
阶段 C	上线拍照识题	单题拍照、整页识题、图文混合、原题/相似题命中链路	6-9
阶段 D	上线 AI 讲题	年级约束讲解、追问、讲解模板、错因分析初版	5-8
阶段 E	上线错题本与个性化练习	错题本、掌握度、薄弱点推荐、复练计划	6-10

这个分期顺序的核心依据是：结构化过滤和高质量 OCR/入库决定了后续功能的上限；而混合检索、RRF、向量过滤与 payload/filter 索引，则决定了召回效率与 result quality 的下限。Elastic 推荐用 RRF 融合全文检索和向量检索；Qdrant 与 Weaviate 也都将“向量 + 关键词/稀疏检索 + 结构化过滤”作为标准能力。⁵

数据模型与自动入库

题库项目的数据模型必须同时服务于**审核、检索、讲题、错题本与推荐**。因此主表不能只存“题干、答案、解析”，还必须显式保存**年级、学科、题型、知识点、难度、来源、版本、章节、审核状态与多模态引用**。同时，自动入库应保留“候选题”和“正式题”两层，以便回溯和纠错。

核心表	关键字段	说明
raw_file	id, file_name, file_type, source_channel, storage_uri, sha256, uploaded_by, upload_time, parse_status, parse_error	原始文件登记表
ingest_job	id, raw_file_id, stage, status, started_at, ended_at, retry_count, metrics_json	入库任务执行记录
question_candidate	id, raw_file_id, page_no, bbox_json, raw_text, raw_image_uri, ocr_json, split_rule, extracted_json, confidence, review_status	AI 切题/抽取后的候选题
question	id, subject, grade, semester, textbook_version, chapter, question_type, knowledge_points_json, difficulty, stem, options_json, answer, analysis, solution_steps_json, has_image, image_uri, source_file_id, source_page, review_state, published_at, updated_at	正式题库主表
question_chunk	id, question_id, chunk_type, chunk_text, chunk_order, embedding, metadata_json	用于检索与向量召回的切片表
review_log	id, candidate_id, reviewer_id, action, before_json, after_json, reason, created_at	审核日志
student_wrong_book	id, student_id, question_id, first_wrong_at, last_wrong_at, wrong_count, latest_answer, error_type, mastery_before, mastery_after, status	错题本
student_mastery	id, student_id, subject, grade, knowledge_point_id, mastery_score, exposure_count, right_count, wrong_count, updated_at	知识点掌握度
search_event	id, user_id, query_text, filters_json, clicked_question_id, result_count, latency_ms, created_at	搜索日志
explain_event	id, user_id, question_id, mode, prompt_version, model_name, success, latency_ms, token_usage_json, rating	讲题日志

建议的题目 JSON 规范如下。它既适合作为 `question` 的 API DTO，也适合作为 LLM 抽取的 Schema 目标。

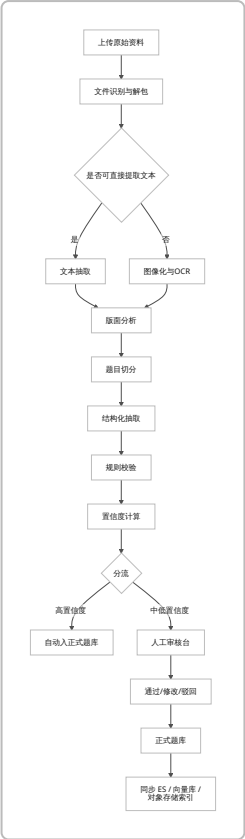
```

{
  "question_id": "math_g3_app_000001",
  "subject": "数学",
  "grade": "三年级",
  "semester": "上册",
  "textbook_version": "人教版",
  "chapter": "多位数乘一位数",
  "question_type": "应用题",
  "knowledge_points": [
    "两位数乘一位数",
    "乘法应用题"
  ],
  "difficulty": "中等",
  "stem": "小明每天读书24页，连续读了6天，一共读了多少页？",
  "options": [],
  "answer": "144页",
  "analysis": "总页数=每天读的页数×天数，即 $24 \times 6 = 144$ 。",
  "solution_steps": [
    "从题目中找出已知条件：每天24页，读了6天。",
    "要求总页数，用乘法。",
    "列式： $24 \times 6 = 144$ 。",
    "答：一共读了144页。"
  ],
  "source": {
    "file_id": "raw_20260701_001",
    "page_no": 12,
    "bbox": [108, 334, 1720, 1016]
  },
  "review": {
    "status": "approved",
    "confidence": 0.94,
    "reviewer_id": "rv_1024"
  }
}

```

自动入库流水线应采用“先文档理解，再结构化抽取，再规则判定，最后分流审核”的思路。PaddleX 官方文档表明，其通用版面解析产线包含版面区域分析、通用 OCR 子产线，以及可选的表格识别和公式识别；PaddleOCR/PP-OCRv5 对简体中文、繁体中文、手写、竖排、拼音和生僻字场景都有支持。对于混合来源文档，Azure Document Intelligence 的 Read 模型支持 PDF、图片以及 Word/Excel/PowerPoint/HTML，并能输出段落、行、词、位置等结构；Google Cloud Vision 的 `DOCUMENT_TEXT_DETECTION` 则适合密集文档文本，并返回 page/block/paragraph/word 层次结构。

6



上面的流程适合小学题库，因为试卷、课本、老师资料 and 手机拍照图片都是典型的“复杂文档”。PaddleX 明确把版面解析定义为“从复杂文档图像中提取结构化信息”的技术，并将其流程拆为版面分析、元素分析和数据格式化；也明确支持表格和公式识别，这对于数学题和带表格题目非常重要。 7

自动入库的规则可以压缩为五类：

环节	规则建议
OCR 规则	对扫描件先做方向分类、去噪、倾斜矫正；记录字符级/行级置信度；手写和印刷混合时分层保存 OCR 结果
切题规则	优先按版面块、题号模式、答案区、题型标题联合切分；整页题单必须保持题号与答案映射
抽取规则	使用固定 JSON Schema； <code>grade/subject/question_type/difficulty</code> 必须来自枚举；知识点必须映射到标准知识点树
校验规则	<code>stem/answer</code> 非空；选择题必须有选项；判断题答案必须落在固定集合；数学基础算式题可做程序验算；图形题若 <code>has_image=true</code> 则图片引用不能为空
置信度规则	综合 OCR 置信度、字段完备度、枚举合法率、知识点映射成功率、程序验算结果、是否命中重复题、模型自评不确定度

置信度建议采用简单可解释的线性或分段评分，而不是一开始就做复杂学习排序。推荐阈值如下：`>=0.92` 自动入库，`0.75-0.92` 进入人工确认，`<0.75` 强制人工审核。对于中间分段，应该在审核台展示**原图 + OCR 文本 + 抽取 JSON + 差异提示**。OpenAI Structured Outputs 可以显著降低“字段缺失、枚举非法、JSON 无法解析”这类问题，因此很适合用在“候选题结构化抽取”环节。 8

检索与拍照识题

小学题库检索的设计原则应当是：**元数据先行，关键词检索为主，向量检索为辅，重排用于最终质量收口。**

Elastic 官方文档明确说明，`filter` 运行在过滤上下文中，不计算相关性得分，适合 `keyword`、日期、数值等结构化字段；同时 `bool.filter` 会受益于缓存与更低的 CPU 开销。对于题库而言，这正好对应“年级、学科、题型、难度、教材版本、章节”等 must-have 条件。 ⁹

因此，检索应分为四层：

1. **过滤层**：年级、学科、题型、知识点、难度、教材版本、章节。
2. **全文层**：题干、答案、解析、标签、别名知识点。
3. **语义层**：相似题、变式题、模糊表达召回。
4. **重排层**：融合过滤约束、关键词命中、语义相似、去重与业务规则。

Elastic 同时支持 `dense_vector`、kNN、kNN 预过滤和 RRF 融合，适合把“关键词 + 结构化过滤 + 向量”放在一个统一的搜索层中；如果希望把向量层拆出去，Qdrant 提供 payload 过滤与 hybrid query，Weaviate 也提供 BM25 + vector 的 hybrid search 及融合权重，Milvus 则更善于大规模向量数据。 ¹⁰

题库最基础、也最关键的 Elasticsearch 查询如下：

```
POST /question_index/_search
{
  "size": 20,
  "query": {
    "bool": {
      "must": [
        {
          "match": {
            "stem": "乘法应用题 读书 24页 6天"
          }
        }
      ],
      "filter": [
        { "term": { "subject": "数学" } },
        { "term": { "grade": "三年级" } },
        { "term": { "question_type": "应用题" } },
        { "terms": { "knowledge_points": ["两位数乘一位数", "乘法应用题"] } },
        { "term": { "difficulty": "中等" } }
      ]
    }
  }
}
```

这个查询范式直接对应 Elastic 的 `bool.query + filter` 机制：`must` 负责文本相关性，`filter` 负责结构化约束，并且 `filter` 不参与打分。对小学题库来说，这种模式比“直接向量搜全库”更稳，因为它先杜绝了跨年、跨题型、跨学科的误召回。 ⁹

混合检索则推荐用 Elastic 的 RRF retriever，将全文召回和向量召回融合：

```
GET /question_index/_search
{
  "retriever": {
    "rrf": {
      "retrievers": [
        {
          "standard": {
            "query": {
              "bool": {
                "must": [
                  {
                    "multi_match": {
                      "query": "三年级 乘法应用题 连续读了6天",
                      "fields": ["stem^3", "analysis", "knowledge_points"]
                    }
                  }
                ],
              }
            },
            "filter": [
              { "term": { "subject": "数学" } },
              { "term": { "grade": "三年级" } },
              { "term": { "question_type": "应用题" } }
            ]
          }
        },
        {
          "retriever": {
            "knn": {
              "field": "stem_vector",
              "query_vector": [0.012, -0.233, 0.441, 0.109],
              "k": 50,
              "num_candidates": 200
            }
          },
          "weight": 1.2
        }
      ],
      "rank_constant": 60,
      "rank_window_size": 100
    }
  }
}
```

这个例子遵循 Elastic 官方的 RRF retriever 语法：`standard` 与 `knn` 可被同一个 RRF retriever 融合，并支持设置权重；Elastic 也明确推荐用 RRF 做 hybrid search。¹¹

如果向量层使用 Qdrant，最实用的用法是 **payload 过滤 + 向量搜索**，并为用于过滤的字段创建 payload index：

```
POST /collections/question_points/points/query
{
  "query": [0.2, 0.1, 0.9, 0.7],
  "filter": {
    "must": [
      { "key": "grade", "match": { "value": "三年级" } },
      { "key": "subject", "match": { "value": "数学" } },
      { "key": "question_type", "match": { "value": "应用题" } }
    ]
  },
  "limit": 20
}
```

Qdrant 官方文档说明，`filter` 会把搜索限制在满足条件的 points 上；为了获得更好的过滤性能，应提前为会用于过滤的 payload 字段创建索引。Qdrant 的 hybrid query 则通过 `prefetch` 并行执行 sparse 与 dense 检索，再用 RRF 或 DBSF 融合。¹²

拍照识题的推荐链路应是：**图像预处理** → **OCR/版面识别** → **题目切分** → **原题精确匹配** → **关键词检索** → **向量召回** → **重排** → **标准解析/讲解生成**。Google Cloud Vision 的 `DOCUMENT_TEXT_DETECTION` 和 Azure Document Intelligence 都支持密集文档文本与手写识别；PaddleX 则更适合中文试卷、书本、扫描件和公式混排场景。对小学题库而言，图片相似度仅适合图形题、几何题、统计图题和整页版式定位，不适合替代 OCR 后的文本检索。¹³

AI讲题、错题本与质控闭环

AI讲题必须建立在**标准答案与标准解析优先**的原则上。也就是说，模型的职责不是“自由发挥解题”，而是根据正式题库中的 `answer / analysis / solution_steps / 年级标签 / 题型标签 / 知识点标签` 生成适龄讲解。这样才能避免小学题使用超纲方法，也更利于错误归因和教学口径统一。

一个合格的 AI 讲题系统至少要支持四种模式：**直接讲解**、**苏格拉底式追问**、**只提示下一步**、**给家长的教学提示**。同一道题在三年级和六年级、在“第一次学”与“复习错题”两种状态下，解释密度应不同。因此讲题服务不仅要输入 `question_id`，还要输入 `student_grade_context`、`explain_mode` 与 `mastery_state`。OpenAI 的 Responses API 支持图像与文本输入，Structured Outputs 可以固定讲解结构；在需要长时后台任务时，还可以使用 background response 模式。¹⁴

错题本不是存一张“错题列表”就结束，它至少要具备三种能力：**错题收录**、**错因归类**、**复练调度**。错因建议从一开始就设定成有限枚举，例如：

`审题错误 / 计算错误 / 概念不清 / 单位错误 / 步骤缺失 / 粗心错误 / 不会列式 / 图意理解错误`。这些错因不一定全部由 LLM 直接判定，更稳妥的方式是“**规则先判断，模型再补充**”：例如口算题优先做程序验算，单位题优先做单位字段检查，列式题优先做表达式解析，剩余复杂情况再交给模型判断。

个性化练习建议从最小可用闭环做起，不要一开始就上复杂推荐算法。一个务实的练习推荐公式可以是：

`推荐分 = 知识点薄弱度 × 题型匹配度 × 难度适配度 × 近期曝光惩罚 × 新鲜度`

其中，知识点薄弱度由 `student_mastery` 维护；题型匹配度和难度适配度由规则控制；近期曝光惩罚用于避免反复推同一道题。这样做的优点是可解释，且方便老师和教研调整。随着数据积累，再引入 bandit、IRT 或更复杂的 mastery tracing。

人审 workflow 必须深度嵌入产品，而不是只做成一个“管理后台”。推荐的人在环闭环如下：

节点	人机分工
候选题生成	机器负责 OCR、切题、抽取、打标签与打分
审核分流	机器按置信度和规则异常分流；人只看高风险样本
审核界面	人查看原图、OCR、结构化 JSON、相似题候选、系统警告
纠错回流	人修改后写入 <code>review_log</code> ，并回灌训练/规则集
线上讲题质检	机器抽样，人工复核；高风险题型建立白名单模板
搜索反馈利用	点击、收藏、复练效果进入 ranking 与推荐回路

对于 LLM 输出，必须建立质量评估。OpenAI 的评估最佳实践文档强调，生成式系统具有非确定性，因此需要显式设计 eval objective、评估数据集与成功指标，而不能只依赖传统软件测试。题库项目建议至少维护四类离线评测集：**结构化抽取集、搜索相关性集、讲题质量集、错因归类集**。 15

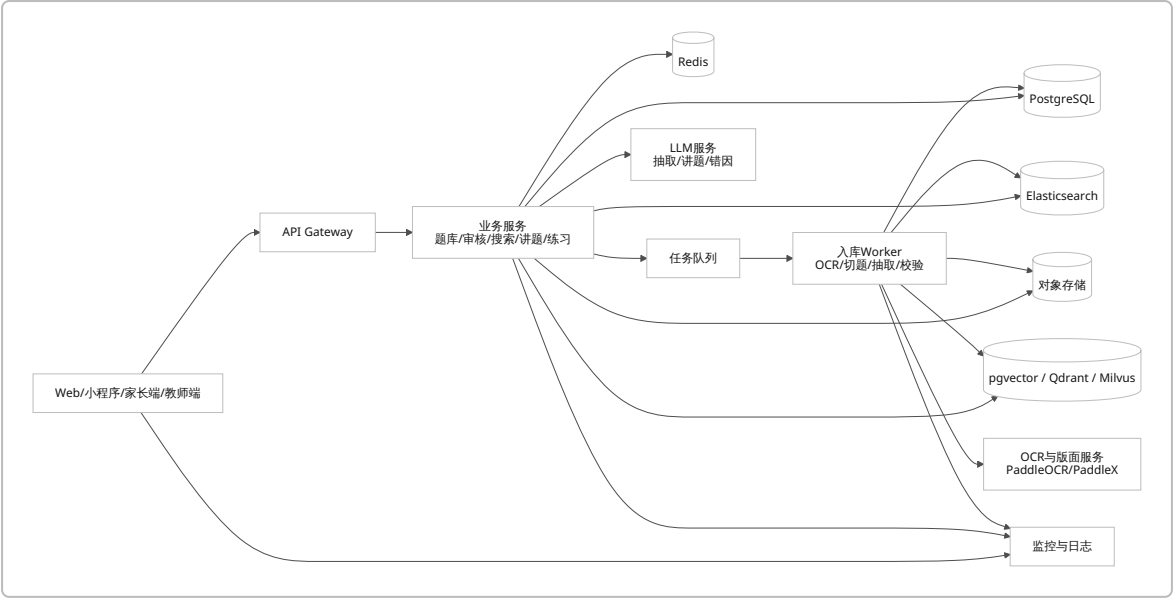
API 与界面建议统一围绕“上传、审核、检索、讲解、练习”五大类设计：

接口/界面	方法	输入	输出
<code>/files/upload</code>	POST	文件、来源、标签候选	<code>raw_file_id</code> 、解析任务 ID
<code>/ingest/jobs/{id}</code>	GET	任务 ID	当前阶段、状态、错误、统计
<code>/review/candidates</code>	GET	状态、置信度范围、文件 ID	候选题列表
<code>/review/candidates/{id}</code>	PATCH	修正字段、审核动作	审核结果、写入日志
<code>/questions/search</code>	POST	<code>query + filters + page + sort</code>	题目列表、facet、命中解释
<code>/questions/{id}</code>	GET	题目 ID	题目详情、来源、相似题
<code>/photos/recognize</code>	POST	图片文件、是否整页	识别到的题块、匹配题目候选
<code>/explanations/generate</code>	POST	<code>question_id + mode + student_context</code>	步骤讲解、提示、可追问话术
<code>/wrong-book</code>	GET/POST	学生 ID、过滤条件	错题列表、错因、复习状态
<code>/practice/recommend</code>	POST	学生 ID、约束条件	练习题单

QA 与监控应覆盖入库、搜索和 AI 三条链路。最少要看的指标包括：OCR 字符错误率、切题准确率、字段抽取合法率、高置信度自动通过准确率、搜索 Top-1/Top-5 命中率、拍照识题首屏命中率、AI 讲题人工通过率、错因一致率、P95 搜索延迟、P95 讲题延迟、索引积压深度、审核 backlog、模型 token 成本。值得注意的是，pgvector 官方文档明确建议通过“近似搜索 vs 精确搜索”的对比监控召回效果，这个思路同样适用于任何 ANN 向量搜索。 16

技术选型、部署扩展与工期

基础架构上，建议把系统拆成五个平面：**业务主库、搜索平面、向量平面、文档理解平面、模型服务平面**。下面先给出推荐的组件架构。



这个架构背后的选型建议如下。表中“推荐性判断”是本报告的工程建议；括号中的能力说明来自官方资料。

组件	推荐方案	备选方案	适用判断
业务主库	PostgreSQL	MySQL	若要在一个系统内同时管理事务数据、JSON 元数据和轻量向量，PostgreSQL 更顺手；pgvector 还能直接复用 Postgres 生态，并支持复制、PITR 与监控。 ¹⁷
关键词检索	Elasticsearch	OpenSearch	ES 的 <code>bool.filter</code> 、 <code>dense_vector</code> 、kNN 过滤与 RRF 已较成熟，适合题库这种“结构化过滤 + 全文 + 向量”的混合场景。 ¹⁸
轻量向量层	pgvector	Weaviate	如果向量量级不大、团队更熟数据库，pgvector 成本最低；还能用 Postgres 客户端直连。 ¹⁷
中型向量层	Qdrant	Weaviate	Qdrant 支持 <code>payload</code> 过滤、 <code>payload index</code> 、 <code>dense+sparse hybrid query</code> ，适合相似题、图文混合检索。 ¹⁹
大规模向量层	Milvus	Weaviate	Milvus 官方强调可扩展到 <code>tens of billions of vectors</code> ，并提供 <code>Lite/Standalone/Distributed</code> 三种部署模式。 ²⁰
OCR 首选	PaddleOCR / PaddleX	Tesseract	面向中文试卷、课本、扫描件与公式混排，PaddleX 的版面解析、OCR、表格与公式识别非常契合。 ²¹

组件	推荐方案	备选方案	适用判断
OCR 云备选	Azure Document Intelligence	Google Cloud Vision	Azure Read 支持 PDF、图片与 Office 文档，且可识别印刷与手写文本；适合作为复杂 Office 文件兜底。 ²²
OCR 手写/密集文档兜底	Google Cloud Vision	Azure Document Intelligence	<code>DOCUMENT_TEXT_DETECTION</code> 返回页/块/段/词层次，并支持手写场景。 ²³
开源 OCR 备用	Tesseract	无	Tesseract 是开源 OCR 引擎，支持 LSTM OCR，但对中文复杂试卷场景通常更适合作为补充而非主力。 ²⁴
LLM 服务	支持图像输入与 Structured Outputs 的多模态模型	本地开源模型	若要做从图片到 JSON 的抽取、讲题与错因分析，图像输入与严格 JSON 输出非常重要。OpenAI 官方文档确认最新模型支持图像输入、多语言与 Structured Outputs。 ²⁵
研发提效	Codex	通用代码助手	Codex 的正确角色是生成接口、数据迁移脚本、审核台代码、评测脚本和测试，不是线上题目解释模型。 ²⁶

如果只给一个**默认推荐栈**，我会建议：

- **主库**：PostgreSQL
- **检索**：Elasticsearch
- **向量**：MVP 用 pgvector；到 50 万题以上再评估 Qdrant 或 Milvus
- **OCR**：PaddleOCR/PaddleX 为主，Azure Document Intelligence 作为 Office 文档兜底
- **模型**：支持图像输入、嵌入和 Structured Outputs 的多模态模型；讲题与抽取分模型配置
- **缓存与队列**：Redis + Kafka/RabbitMQ（或云原生消息队列）
- **对象存储**：S3 兼容存储保存原图、切题图、审计材料

部署时应把“成本敏感”和“体验敏感”分开：

自动入库、批量 OCR、历史回灌、向量重建都属于**后台吞吐任务**，适合异步队列与可中断 Worker；搜索、拍照识题和讲题则属于**前台时延敏感链路**。OpenAI 官方文档也区分了同步 Responses 与 background response 两种处理模式；如果抽取任务或批量讲解需要长时运行，后台模式更合适。 ²⁷

在扩展性上，系统应预留四类增长：

1. **数据量增长**：从几十万题升到百万题后，需要把 `question_chunk` 与检索索引独立生命周期管理。
2. **流量增长**：拍照识题和讲题高峰通常集中在晚间，需对 OCR、LLM、搜索层做独立扩缩容。
3. **模型升级**：Embedding 升级、OCR 替换、讲题模型替换，必须支持双索引并行与蓝绿切换。
4. **组织增长**：若未来服务学校/机构，多租户隔离、审计日志、RBAC 与数据导出要尽早纳入设计。

如果项目面向 C 端小学生与家长，部署与产品策略还需要显式处理未成年人合规：数据最小化、监护人同意、儿童个人信息保护规则、投诉举报渠道、与教学无关的信息控制都应在产品层体现，而不仅仅是法务文本。中国相关法规已对“儿童个人信息”“未成年人网络保护”“在线教育不得推送与教学无关信息”等作出明确要求。 ²⁸

最后给出一份可直接执行的阶段工期表。这里的工期是**工程建设工期**，不包含大规模历史资料清洗与长期教研标注。

阶段	主要任务	关键风险	通过标准	估算人周
MVP	上传、OCR、切题、抽取、审核台、正式题库、ES 检索	混合格式文本质量不足	可稳定入库，审核台可回写，按元数据检索可用	10-14
阶段 A	表格/公式、重复题、置信度体系、批量回灌	数学题公式 OCR 误差	自动通过样本准确率达到可运营水平	6-8
阶段 B	向量召回、混合检索、重排、相似题	召回噪声与成本上涨	Top-5 命中率明显优于纯关键词	5-7
阶段 C	单题拍照、整页识题、图片切题	手机拍照质量波动	首屏命中率、识别延迟达到上线要求	6-9
阶段 D	分步讲题、追问、错因分析	幻觉与超纲讲解	人工抽检通过率与家长试用反馈达标	5-8
阶段 E	错题本、掌握度、个性化练习	推荐重复、掌握度失真	复练完成率、回访正确率提升	6-10

综合来看，这个项目最值得投入工程资源的地方，不是“再多接几个模型”，而是三件事：**把题目数据结构化好、把检索约束做对、把人审闭环嵌入系统**。只要这三件事做好，拍照识题、AI讲题、错题本与个性化练习都会有一个可靠的底座；反过来，如果这三件事做不好，再强的模型也只能在低质量题库上放大噪声。 ²⁹

¹ ⁹ ¹⁸ ²⁹ Query DSL | Elastic Docs

<https://www.elastic.co/docs/explore-analyze/query-filter/languages/querydsl>

² ²⁸ 儿童个人信息网络保护规定_中央网络安全和信息化委员会办公室

https://www.cac.gov.cn/2019-08/23/c_1124913903.htm

³ ²⁵ Models | OpenAI API

<https://developers.openai.com/api/docs/models>

⁴ ¹⁵ Evaluation best practices | OpenAI API

<https://developers.openai.com/api/docs/guides/evaluation-best-practices>

⁵ Hybrid search | Elastic Docs

<https://www.elastic.co/docs/solutions/search/hybrid-search>

⁶ ⁷ ²¹ 通用版面解析产线 - PaddleX 文档

https://paddlepaddle.github.io/PaddleX/latest/pipeline_usage/tutorials/ocr_pipelines/layout_parsing.html

⁸ Structured model outputs | OpenAI API

<https://developers.openai.com/api/docs/guides/structured-outputs>

¹⁰ Dense vector field type | Elasticsearch Reference

https://www.elastic.co/docs/reference/elasticsearch/mapping-reference/dense-vector?utm_source=chatgpt.com

¹¹ RRF retriever | Elasticsearch Reference

<https://www.elastic.co/docs/reference/elasticsearch/rest-apis/retrievers/rrf-retriever>

¹² Search - Qdrant

<https://qdrant.tech/documentation/search/search/>

13 23 Detect handwriting in images | Cloud Vision API | Google Cloud Documentation

<https://docs.cloud.google.com/vision/docs/handwriting>

14 API Overview | OpenAI API Reference

<https://developers.openai.com/api/reference/overview/>

16 17 GitHub - pgvector/pgvector: Open-source vector similarity search for Postgres • GitHub

<https://github.com/pgvector/pgvector>

19 Payload - Qdrant

<https://qdrant.tech/documentation/manage-data/payload/>

20 Milvus | High-Performance Vector Database Built for Scale

<https://milvus.io/>

22 Read model OCR data extraction - Document Intelligence - Foundry Tools | Microsoft Learn

<https://learn.microsoft.com/en-us/azure/ai-services/document-intelligence/prebuilt/read?view=doc-intel-4.0.0>

24 Tesseract Open Source OCR Engine (main repository)

https://github.com/tesseract-ocr/tesseract?utm_source=chatgpt.com

26 CLI – Codex | OpenAI Developers

<https://developers.openai.com/codex/cli>

27 Background mode | OpenAI API

<https://developers.openai.com/api/docs/guides/background>